Day 1: Natas 18(Script) & THM: HTTP in Detail tasks 1-4

- Don't trust cookies(cookies can be edited thru devtools etc)
- Small ranges/keyspaces are easily enumerated: possible values can be guessed quickly
- High-entropy random generation(CSPRINGs) for session tokens: guessing is mathematically impossible – real systems need this!
- **HTTPS = Secure version (added TSL/SSL encryption) of HTTP (Set of rules used for communicating with web servers)**
- **A URL may include: Scheme(HTTP), User(Username:Password), Host(Domain name/IP address), Port(Usually 80 for HTTP and 443 for HTTPS but could be any between 1-65535), Path, Query String(/blog?id=1), Fragment(reference to a location on actual page)**
- **GET(info from server), POST(Submitting data to server), PUT(Submitting data to server to update info), DELETE(deleting info/records from a server)**

🕵 DFIR Takeaway: Detection vs. Deep Analysis

- Server Access Logs (The Overview): * What they show: *That* an attack is happening. Automated brute-forcing creates massive spikes in requests from a single IP address.
  - Where to find them: Linux (/var/log/apache2/access.log or /var/log/nginx/access.log) or Windows IIS (C:₩inetpub₩logs₩).
  - The Limitation: Standard logs record requested URLs, but ignore hidden details like modified cookies or POST payloads(body).
- Packet Captures / PCAPs (The Details): * What they show: *How* the attack happened. PCAPs record raw network packets so investigators can see the exact manipulated cookie values and stolen session IDs.
  - How they're used: Recorded using command-line tools like `tcpdump` (`sudo tcpdump -w capture.pcap`) and opened in **Wireshark** to reconstruct full TCP streams and view the exact data sent by the attacker.

Day 2: Natas 19(Script), Natas 20(Script) & THM: HTTP in Detail tasks 5-7

- Use the application section of the devtools to view and analyze cookies, to sense a pattern in things like session ids.
- Once again, cookies can be edited thru devtools. Use this to brute force
- Cyberchef useful to find patterns(like hexcoding). Thus, randomness and entropy is important in cryptography security(ex. CSPRINGs)

DFIR & Threat Hunter Synthesis: Day 2

1. Adversary Tradecraft vs. Detection Opportunities

- **Cookie Tampering:** Attackers manually edit session tokens using DevTools or proxies to hijack accounts without entering credentials.
- **The Forensic Footprint:** While basic logs look like standard requests, threat hunters spot tampering by watching for sudden spikes in valid activity from a single IP or rapid, unexpected changes in session token formats.

2. Forensic Artifact Correlation & Log Types

- **Server Access Logs:**
  - **What they show:** Basic web traffic (IP address, requested page, HTTP status codes like 404 or 403).
  - **DFIR Use:** Spotting high-volume directory scanning, fuzzing, or brute-force attempts.
  - **Limitation:** They miss the contents of HTTP request headers and cookies by default.
- **Application Audit Logs:**
  - **What they show:** Business-logic actions tied to user identity (e.g., who logged in, privilege changes, sensitive file views/downloads).
  - **DFIR Use:** Detecting insider threats, account takeovers, and unauthorized data access that basic web server logs miss.
- **Packet Captures (PCAPs):**
  - **What they show:** Full raw network traffic, including cookie headers and transferred data (when unencrypted or decrypted).
  - **DFIR Use:** Finding simple, hex-encoded sequential session IDs to prove weak token entropy and poor session generation logic.

3. Threat Hunting & Engineering Applications

- **Detection Engineering (SIEM & Logging):**
  - **The Problem:** Basic web logs miss cookie details, making session hijacking hard to spot.
  - **The Fix:** Security engineers configure servers to log session IDs (or hashed versions) into a **SIEM** (centralized log analyzer) or track session state changes in application audit logs to flag suspicious activity—like the same session ID appearing from two different locations at once.
- **Automating Anomaly Detection:** Cyber detectives write Python scripts and SIEM rules to calculate token entropy and baseline normal error rates—triggering automated alerts whenever predictable token sequences or low-entropy patterns appear.

Day 3: Natas 21(Script) & THM DNS In Detail

- You have to ask "what does this share with other things running nearby?" not just "is this code safe on its own?"
- Never write untrusted keys into a trusted data structure
- **Cookies are honor codes, not a real secure wall**
- **Session ID's are identity**
- **Admin.tryhackme.com = Admin: Subdomain(maximum 63 chars), tryhackme: second level domain, .com: TLD(Top Level Domain)**
- **gTLD: purpose(org,com,edu,gov) ccTLD:geographical(ca,co.uk)**
- **A URL may include: Scheme(HTTP), User(Username:Password), Host(Domain name/IP address), Port(Usually 80 for HTTP and 443 for HTTPS but could be any between 1-65535), Path, Query String(/blog?id=1), Fragment(reference to a location on actual page)**

🕵 DFIR Takeaway: Detection vs. Deep Analysis

- Server Access Logs (The Overview): * What they show: *That* an attack is happening. Automated brute-forcing creates massive spikes in requests from a single IP address.
  - Where to find them: Linux (`/var/log/apache2/access.log` or `/var/log/nginx/access.log`) or Windows IIS (`C:₩inetpub₩logs₩`).
  - The Limitation: Standard logs record requested URLs, but ignore hidden details like modified cookies or POST payloads(body).
- Packet Captures / PCAPs (The Details): * What they show: *How* the attack happened. PCAPs record raw network packets so investigators can see the exact manipulated cookie values and stolen session IDs.
  - How they're used: Recorded using command-line tools like `tcpdump` (`sudo tcpdump -w capture.pcap`) and opened in Wireshark to reconstruct full TCP streams and view the exact data sent by the attacker.

Day 4: Natas 22(Script) & THM OSWAP ZAP

- $_GET: array that holds every query parameter from the URL(after ?)
- A redirect header only tells the *browser* to leave; it does nothing to the PHP script still running on the server.
- **header("Location: /");**

  exit; // this line is what actually matters

OWASP ZAP is a free, intercepting-proxy-based tool that sits between your browser and a website so you can see and manipulate the raw traffic between them, letting you find web application vulnerabilities either automatically (spidering + scanning) or manually (intercepting and tampering with real requests). (Before, not after attack)

Threat Hunting, Incident Response (DFIR), & SOC Analysis

- Malicious Traffic Replay: During a cyber investigation, a defender might recover a suspicious HTTP request from server logs or a packet capture (.pcap). The analyst imports that raw request into a proxy to safely replay it in an isolated sandbox, confirming whether the exploit attempt actually succeeded.
- Analyzing Malware Web Communication: Incident responders detonate web-based malware or phishing kits inside an isolated environment routed through an intercepting proxy. This exposes the exact command-and-control (C2) servers, API calls, or exfiltration routes the malware uses.
- Decoding Hidden Web Payloads: Attackers often obscure or encode payloads (e.g., using Base64, URL encoding, or custom obfuscation) inside cookies or headers. Proxies feature built-in decoders that allow analysts to unpack and inspect these payloads instantly.

Day 5: Natas 23 & THM introduction to networking pt 1

- passwd = "11iloveyou"
  strstr("11iloveyou", "iloveyou") → finds the substring "iloveyou" inside it → returns a truthy value ✅
  "11iloveyou" > 10 → PHP converts "11iloveyou" to the number 11 (reading only the leading digits, discarding the rest) → 11 > 10 → true ✅
  = PHP's loose comparison rules (Type Juggling)

When PHP sees "string" > number, it doesn't compare them as strings — it tries to numerically interpret the string. The conversion rule (in PHP versions before 8) is:

- **If the string starts with digits, take just that leading numeric portion (e.g., "11abc" → 11, "3.5xyz" → 3.5).**
- **If the string doesn't start with any digit at all, it converts to 0.**

- **Potential fix: if(strstr($_REQUEST["passwd"], "iloveyou") && is_numeric($_REQUEST["passwd"]) && $_REQUEST["passwd"] > 10)**

OSI: APP -> Decode -> Connect session -> choose protocol -> ip -> MAC/checking data suitability prep for transmission -> Hardware(where pulses sent and received)

Protocols: TCP (Accuracy), UDP(Speed)

As data gets passed down each layer more info about the data is added on(encapsulation), the process in which that gets reversed when received by the second computer(de-encapsulation)

TCP/IP: Application(layer7+6+5 from OSI), Transport, Internet, Network Interface(Layers 1+2 OSI)

TCP/IP is more about a suit of protocols:

- **TCP: connection based, three-way handshake:**

My comp sends a special request to remote server indicating initialization of a connection(SYN bit)

Server responds with a packet containing SYN bit + ACK(acknowledgement) bit

My computer sends a packet that contains ACT by itself

== lossless connection between two computers

🕵️ The Cyber Detective's Core Rule

"Attackers exploit how code is *supposed* to work; detectives investigate how code *actually* behaves."

The Two Rules You Learned Today:

1. Computers are literal, not smart (Natas 23):
    - When code uses loose logic, it makes lazy guesses—like turning "11iloveyou" into the number 11. Attackers look for these lazy shortcuts to sneak past doors. Detectives spot where developers trusted lazy code instead of strict checks.
2. Network connections leave digital footprints (TCP Handshake):
    - Devices can't talk quietly on a network; they *must* introduce themselves first. The 3-way handshake (SYN -> SYN-ACK -> ACK) is the digital equivalent of a handshake before a conversation. If two devices spoke, that handshake happened, and it left a record.

Day 6: Natas 24(Script) & THM Introductory Networking pt 2

- Strcmp() in php returns 0 when identical, but any type can be passed
- If you pass an array and compare it with a string it returns null
- **Never assume user input has the type you expect — a function that's only "safe" when given a string can behave completely differently (and often incorrectly) when given something else, like an array, and attackers can freely send whatever type they want.**
- **Ping: test whether a connection to a remote source is possible(ICMP)**
- **Traceroute: allows seeing every intermediate step between my computer and the server/resource I requested**
- **WHOIS: Information about a domain**
- **Dig: manually query recursive DNS servers for info abt domains**

  (dig <domain> @<dns-server-ip>)

  TTL(Time To Live hops or seconds or time)

🕵️ Day 6 Cyber Detective Takeaway

"Attackers look for where input validation stops, and network diagnostics tell you *how* traffic actually moves."

Key Principles You Mastered Today:

- **Type Safety Over Assumptions:** You learned that in web auditing, you can never trust the data type sent by a client. Functions like `strcmp()` fail unsafe when given unexpected structures (like an array instead of a string), creating instant logic bypasses.
- **Network Visibility (Ping, Traceroute, DNS):** You now know how to map out pathways and query infrastructure directly:
  - `ping` checks reachability via ICMP.
  - `traceroute` / `tracert` maps the path router-by-router using TTL mechanics.
  - `whois` & `dig` gather domain ownership and query DNS records straight from authoritative/recursive servers.

🧮 Summary Cheat Sheet

Tool / Concept	Core Function / Mechanism	Detective Note
`strcmp()` **Array Trick**	Array input returns `NULL`, evaluating to `true` under loose check `!strcmp()`	Always enforce `is_string()` or strict equality (`===`)
`traceroute`	Manipulates TTL values to provoke ICMP "Time Exceeded" errors	Windows uses ICMP; Linux uses UDP (or TCP with `-T`)
`dig`	Queries DNS servers directly (`dig <domain> @<server>`)	Bypasses local DNS caching to inspect raw zone responses
TTL (Time To Live)	Hop limit for packets / cache duration for DNS	Prevents infinite routing loops and controls record expiration

IP gets the packet to the right building (host).

TCP/UDP gets the packet to the right apartment door (port number).

Application Protocols (HTTP, DNS) are the actual conversations taking place inside the apartment.

ICMP is the mail carrier telling you the address doesn't exist or the mailbox is full.

Day 7: Natas 25(Script) & THM: Active Recon

- **You can put code anywhere(<?php... ?>) on any file(ex. Log), and the include function would run that code(why log poisoning is important)**
- **Single-pass filter isn't security**
- **Attacker can get their input into any file & find a way to make the server include/execute that file → full remote code execution**
- **Traceroute sends multiple packets with starting TTL values of 1 then 2 then 3 until it reaches the target computer to get the intermediate router info(IP, etc) – sends udp by default(-T for TCP, -I for ICMP)**
- **Telnet communicate w a remote system via command line (cleartext, so secure alternative os SSH): allows banner grabbing → connect to a service and read the initial response that has info.**
- **Netcat: TCP/UDP. Can function as a client or a server**
- **Nc –vnlp <port> : server**

Quick Reference

Command	Example
ping	`ping -c 10 10.64.157.41` on Linux or macOS
ping	`ping -n 10 10.64.157.41` on Windows
ping (IPv6)	`ping -6 MACHINE_IPV6` or `ping6 MACHINE_IPV6`
traceroute	`traceroute 10.64.157.41` on Linux or macOS
tracert	`tracert 10.64.157.41` on Windows
traceroute (IPv6)	`traceroute -6 MACHINE_IPV6` or `traceroute6 MACHINE_IPV6`
mtr	`mtr 10.64.157.41` for real-time path monitoring
telnet (legacy)	`telnet 10.64.157.41 PORT_NUMBER`
netcat as client	`nc 10.64.157.41 PORT_NUMBER`
netcat as server	`nc -lvnp PORT_NUMBER`
netcat (IPv6)	`nc -6 MACHINE_IPV6 PORT_NUMBER`

Command	Example
curl for HTTP banner	`curl -I http://10.64.157.41` or `curl -I https://10.64.157.41`

🕵️ Day 7 Cyber Detective Takeaway: "Input Boundaries & Active Telemetry"

"Attackers use active probes and input boundaries to force systems into revealing their internal behavior; detectives analyze those exact signals to map the attack surface."

1. Adversary Tradecraft vs. Detection Opportunities

- **Log Poisoning & LFI (Natas 25):**
  - **The Tradecraft:** Attackers inject executable code (like `<?php ... ?>`) into non-code files (like HTTP `User-Agent` headers recorded in access logs) and use Local File Inclusion (LFI) to trick the application into executing it.
  - **The Forensic Footprint:** Look for unusual string patterns (e.g., PHP execution tags or directory traversal paths `....//`) inside web server access logs, paired with unexpected process creation from the web server user (`www-data`).
- **Active Reconnaissance & Service Fingerprinting:**
  - **The Tradecraft:** Attackers use tools like `traceroute`, `telnet`, and `netcat` to map network paths, identify live hosts, and grab service banners.
  - **The Forensic Footprint:** Active probing leaves clear network telemetry—such as bursts of incremental TTL packets, unexpected ICMP "Time Exceeded" errors, or raw banner requests on common ports (80, 443, 22).

Day 8: Natas 26(Script) & THM NMAP part 1

- Deserializing untrusted data isn't just "risky" in the abstract — if any class with a magic method (__destruct, __wakeup) exists anywhere in the codebase, an attacker who controls the serialized input can weaponize that method using properties they fully control, often without ever needing to call any function directly themselves.
- **NMAP = port scanning**
- **ICMP: layer3(no ports, just checking if target host is turned on)**

TCP/UDP: transport layer(layer 4)

- **TCP: three way handshake to each target port(default of no sudo permission**
- **SYN: interrupts the handshake after the second part(SYN-ACK from server) – default in nmap(if sudo permissible)**
- **UDP: connectionless: no response(open|filtered), response(very rare, open), Closed(ICMP packet returned)**

1. ICMP(ping) Traffic (Stops at Layer 3)

- **Because ICMP is a Layer 3 protocol alongside IP, it does not have port numbers.**
- **When a target receives an ICMP Ping (Echo Request):**
- **Layer 1: Network Card catches the physical bits.**
- **Layer 2: OS checks the destination MAC address.**
- **Layer 3: OS reads the IP header and sees the protocol type is ICMP.**
- **Processing Stops Here: The operating system kernel processes the ICMP request directly and sends back an ICMP reply. It never passes the packet up to Layer 4 because there are no ports to check.**

2. TCP/UDP Scans (-sT, -sS, -sU) (Goes up to Layer 4)

- TCP and UDP live at Layer 4. Their job is to bridge the OS network stack to specific running applications using port numbers.
- When a target receives a TCP SYN or UDP packet:
- Layer 1: Network Card catches the physical bits.
- Layer 2: OS verifies the MAC address.
- Layer 3: OS verifies the target IP address, strips the IP header, and discovers a Layer 4 payload inside (TCP or UDP).
- Layer 4: OS inspects the Destination Port Number (e.g., Port 80) to check if an application (like Apache or NGINX) is actively listening.

- SYN → RST = TCP closed port
- SYN → ACK/SYN = TCP open port
- NULL/FIN/Xmas scans: used for firewall evasion as they don't send SYN flag set like TCP(which gets firewalled), expects same results as UDP in open or open|closed, usually RST if closed

🔍 Day 8 Detective Summary

1. Web Forensics: Deserialization

- **The Vulnerability:** Attackers do not need to invoke dangerous functions directly—injecting serialized objects manipulates object properties, triggering native PHP "magic methods" (__destruct()) during memory cleanup.
- **Detective Mindset:** Inspect code for unverified unserialize() calls and monitor web logs/traffic for serialized string patterns (e.g., O:4:...) that attempt unauthorized file writes or execution.

2. Network Telemetry: Layer 3 vs. Layer 4

- **ICMP (Layer 3):** Processed entirely by the operating system kernel. It verifies host presence without interacting with application port sockets.
- **TCP/UDP (Layer 4):** Connects to specific running applications via ports.
    - **TCP Connect (-sT):** Completes the 3-way handshake, generating standard application logs.
    - **SYN Scan (-sS):** Aborts with a RST before completion, avoiding application logs but leaving connection attempt traces in network firewalls.
    - **Evasion Scans (NULL/FIN/Xmas):** Omit standard SYN flags to bypass stateless firewall rules. Detectable via anomalous TCP flag combinations in NIDS/PCAP telemetry.

Day 9: Natas 27(Script) & THM Nmap pt 2

- Test empirically instead of trusting a single expected path.
- Php disregards trailing spaces
- Multiple independently "safe-looking" defenses — input escaping, a whitespace check, and a length limit — combined in an order the developer didn't anticipate, and the underlying database's own quirky comparison behavior turned that combination into a way to leak another account's data.

NSE: Script library in NMAP(written in Lua language)

Categories:

- **safe:- Won't affect the target**
- **intrusive:- Not safe: likely to affect the target**
- **vuln:- Scan for vulnerabilities**
- **exploit:- Attempt to exploit a vulnerability**
- **auth:- Attempt to bypass authentication for running services (e.g. Log into an FTP server anonymously)**
- **brute:- Attempt to bruteforce credentials for running services**
- **discovery:- Attempt to query running services for further information about the network (e.g. query an SNMP server).**

Windows host will block all ICMP(ping) requests via firewall. This means Nmap, which uses ping to establish the activity of the target, will consider such host dead and not scan it at all. So –Pn allows Nmap to not bother the pinging the host before scanning it(consider it alive) → eats up more time.

Click enter to see the progress in scan

-vv: very verbose: good for explanations and finding things

🕵️‍♂️ Day 9: Cyber Detective Field Notes

1. Investigation Mindset: Empirical Validation Over Assumptions

- **The Concept:** Multiple individual defenses (escaping, whitespace filtering, string length limits) can fail when combined out of order or passed to an underlying engine (e.g., SQL database string truncation).
- **Detective Application: Never trust input validation at face value.** When investigating authentication bypasses or account takeovers, test inputs empirically across the entire application-to-database pipeline. Look for trailing whitespace (%20), truncated payloads, or unexpected string comparison quirks in backend logs.

2. Threat Analysis: Decoding Intent via Nmap Scripts (NSE)

- **The Concept:** Nmap Scripting Engine categories range from harmless enumeration (safe, discovery) to aggressive actions (auth, brute, vuln, exploit).
- **Detective Application: Categorize telemetry to identify attacker phases.** In SOC/SIEM logs:
    - Probing with auth or brute scripts indicates **Credential Stuffing / Active Reconnaissance**.
    - Probing with vuln or exploit scripts indicates an **Imminent Exploitation Attempt**.

3. Network Reconnaissance: Uncovering Silent Targets & Evidence

- **The Concept:** Windows Firewalls drop ICMP ping requests by default, making active machines look dead to standard discovery scans unless -Pn is used.
- **Detective Application:** * **Network Mapping (-Pn):** When threat hunting inside a compromised internal network, force -Pn to locate hardened, "silent" systems that refuse ICMP probes.
    - **Forensic Evidence (-vv):** Use high verbosity to capture granular packet flags and timing data needed to prove network state and document timeline findings in formal incident reports.

Day 10: Natas 28(Script) & THM Traffic analysis essentials pt 1

- ECB: encryption system dividing text up to a certain number of bytes(Ex. 16), into blocks and encrypting independently using a same key. Thus the result is always the same. Needs to be exact multiples of 16 bytes so uses padding.
- The Vulnerability: Because ECB encrypts identical plaintext blocks into identical ciphertext blocks, it reveals structural patterns in encrypted traffic even without knowing the secret key.
- The Vulnerability: ECB lacks integrity checks or block chaining. Individual encrypted blocks can be moved, swapped, or replaced without breaking the decryption process of the adjacent blocks.
- The Vulnerability: Systems using ECB leak repetitive data sequences across network captures and database records.
- I plaintext = 1 byte

SIEM: threat detection, compliance, incident management thru available data(logs traffic stats)

SOAR: coordinate and automate taska between people, tools, data to identify anomalies threats vulnerabilities

SIEM continuously ingests data and identifies suspicious patterns.

SIEM hands the resulting alert to SOAR.

SOAR runs an automated response workflow, freeing analysts up to focus on complex, high-level threat hunting.

Address Resolution Protocol (ARP) is the local network's phonebook. It maps an IP address (e.g., 10.10.99.199) to a physical hardware MAC

address (e.g., 00:1A:2B:3C:4D:5E). – operates at layer 2(MAC, Data Link), and firewalls are layer 3/4

🕵️‍♂️ Day 10: Cyber Detective Field Notes

1. Cryptography: ECB Structural Leakage & Bit-Flipping

- The Concept: Electronic Codebook (ECB) encrypts 16-byte blocks independently with the same key. Identical plaintext yields identical ciphertext, requiring padding (where 1 char = 1 byte).
- Detective Application: Pattern recognition bypasses encryption. Look for repeating 16-byte hex blocks in DB dumps or PCAPs to spot structural patterns. Because ECB lacks block chaining, attackers can splice, swap, or delete blocks to elevate privileges without knowing the key.

2. SOC Architecture: SIEM, SOAR & The Human Cyber Detective

- The Concept: SIEM centralizes logs to detect threats; SOAR uses playbooks to automate responses.
- Detective Application: Automation handles the routine; humans handle the complex.
    - SIEM (The Eyes): Ingests and correlates raw log data to detect suspicious patterns.
    - SOAR (The Hands): Executes automated response workflows (isolating hosts, blocking IPs) to mitigate high-fidelity alerts instantly.
    - The Human Cyber Detective (The Brain): Investigates edge cases, validates false positives, hunts for novel/unseen threats (zero-days), refines automation playbooks, and conducts deep forensic analysis when automated rules fall short.

3. Traffic Analysis & OSI Layer Containment

- The Concept: ARP operates at Layer 2 (mapping IPs to MACs), while traditional firewalls filter at Layers 3/4 (IPs and Ports).
- Detective Application: Firewalls contain damage; switches fix Layer 2. When an IDS flags an IP for ARP spoofing, a firewall cannot stop the Layer 2 broadcast. Instead, you block the attacker's active TCP/UDP ports (e.g., SSH 2222 or C2 4444) on the firewall for immediate containment, while the network team enables Dynamic ARP Inspection (DAI) on switches for remediation.

Day 11: Natas 29(Script)

Wildcards are commonly used in shell commands in Linux and other Unix-like operating systems. It is a character that can be used as a substitute for any of a class of characters.

For example, a question mark (?) can be used as a wildcard character in shell commands to represent exactly one character.

While a star wildcard — symbolised by an asterisk (*) — can represent all single characters or any string.

🕵️ Day 11: Cyber Detective Field Notes (Code Auditing & Sink Analysis)

1. Root Cause Analysis: Sinks Over Filters

- **The Forensics Concept:** Attackers rarely try to beat strong filters directly; they look for weak **sinks**—the backend functions that actually process user input (like Perl's 2-argument `open()`).
- **Detective Mindset:** When auditing source code, don't just ask *"What input does this code block?"* Ask **"Where does this input eventually land, and does that function treat raw data as control commands?"**

2. Shell Context & Metacharacters

- **The Forensics Concept:** Whenever an application passes untrusted input into an underlying shell interpreter (`/bin/sh`), special metacharacters drastically alter how the command line is executed:
  - **Comments (#):** Tell the interpreter to ignore appended suffixes (like extra extensions added by backend code).
  - **Wildcards (*, ?):** Allow path traversal and command execution while completely bypassing string-matching filters (blacklists).
- **Detective Mindset:** In log analysis or PCAP forensics, watch for unusual characters (`|`, `#`, `?`, `*`) in URL parameters—they indicate command execution or filter-evasion attempts.

3. Forensic Remediation: Strict Separation of Code & Data

- **The Forensics Concept:** Security vulnerabilities happen when data (user input) and instructions (commands) are mixed in the same string.
- **The Fix:** Modern defenses mandate strict separation:
  - Use **3-argument function calls** (e.g., `open(my $fh, '<', $filename)`) so the interpreter forces user input to stay pure data.
  - Avoid implicit shell execution entirely by using parameterized APIs or strict allowlisting (only accepting expected characters like alphanumeric strings).

Day 12: Natas 30(Script) & THM: Snort(part 1)

- Perl's quote() function takes in a secret second argument(a data type example(ex. 4 is an integer)
- By doing this it bypasses the escaping filter
- → submit an array of values as an injection([INJECTION, Integer])
- How a function is supposed to be used != everything the function can do
- **Intrusion Detection system: alert**
- **Intrusion Prevention System: terminate**
- **Snort: use cases**
  - **Packet Sniffer: Read and display network traffic packets**
  - **Packet Logger: Save network traffic data to the disk**
  - **Full-Blown NIPS(Network intrusion prevention system): Inspect network traffic and stop malicious packets(terminate the connection)**

Rules in Snort: specific conditions(protocol TCP/UDP/ICMP, Source/destination IP addresses, target port numbers, text/hex sequences within a packet's payload, and an action to take(alert, log, drop)

Flag	What it adds
-v	IP/port summary
-d	+ payload data
-e	+ MAC addresses
-X	full raw hex of everything
-i	which interface to sniff on

Snort needs superuser (root) rights to sniff the traffic

1. Attackers Abuse Functionality, Investigators Trace Intent

In Natas 30, the exploit didn't use a broken feature—it used Perl's quote() function exactly as coded, but in a way the developer never anticipated.

- **The Rule:** Never assume a system is secure just because it uses standard, built-in functions. When analyzing a breach for the Bureau, you aren't just looking for broken code; you are looking for how an adversary repurposed valid system behavior to bypass controls.

2. Control Your Forensic Resolution

Snort's flags (-v, -d, -e, -X) represent different levels of evidence gathering.

- **The Rule:** Collecting network data is about balance. If you capture only headers (-v, -e), you prove *who* talked to *whom*, but you can't prove *what* was said. If you capture full hex dumps (-X), you get absolute proof of the payload, but you generate massive amounts of data that take time and storage to analyze. The best forensic investigators know exactly how much data to capture for the specific question they are trying to answer.

3. Know Where the Traffic Died (Passive vs. Inline)

Understanding the operational difference between an **IDS** (passive) and an **IPS** (inline) determines how you build your timeline.

- **The Rule:** An IDS alert proves an attack packet was **sent and observed**. An IPS alert proves an attack packet was **detected and blocked**. In court, confusing the two means the difference between proving a system was successfully compromised and proving an attack was stopped at the front door.

Day 13: Natas 31(Script) & THM Snort pt2

- Send in two different files(two tuples: one the string 'ARGV' and second a csv file)
- When a perl script gets this list as input param('file'), it takes the first value(String "ARGV")
- Special perl situation: <$file> reads the literal string ARGV and it activates a different mode. It reads whatever files listed in the built-in array @ARGV, which takes URL's query string that don't look like normal key=value pairs and dumps those keywords staright into @ARGV.

IDS mode:

Parameter	Description
-c	Defining the configuration file.
-T	Testing the configuration file.
-N	Disable logging.
-D	Background mode.
-A	Alert modes; **Full:** Full alert mode, providing all possible information about the alert. This mode is also mode, Snort defaults to this mode. **Fast mode displays the alert message, timestamp, source and destination IP addr** **Console:** Provides fast style alerts on the console screen. **cmg:** CMG style, basic header details with payload in hex and text format. **None:** Disabling alerting.

Snort IPS mode is activated with the -Q-- daq afpacket parameters

- alert: Generate an alert and log the packet.
- log: Log the packet.
- drop: Block and log the packet.
- reject: Block the packet, log it, and terminate the packet session.

Snort Rule Anatomy

Example:

```
alert ip any any <> any any (msg:"IP ID Found"; id:35369; sid:1000001; rev:1;)
```

Rule Header Components

A. Action

- `alert`: Log packet and raise notification (IDS mode).
- `log`: Log packet silently without generating an alert.
- `drop`: Block/drop packet inline (IPS mode).
- `reject`: Block packet, log it, and terminate session (TCP RST / ICMP unreachable).

B. Protocol

- `ip`: All IP traffic regardless of protocol.
- `tcp`: Transmission Control Protocol (HTTP, SSH, FTP).
- `udp`: User Datagram Protocol (DNS, DHCP, SNMP).
- `icmp`: Internet Control Message Protocol (Ping, errors).

C. Source & Destination IP Addresses

- `any`: Matches any IP address.
- `192.168.1.56`: Single host IP.
- `192.168.1.0/24`: Subnet / IP range.
- `!192.168.1.0/24`: Negation (!) — match anything *except* this range.
- `[192.168.1.0/24, 10.0.0.0/8]`: Array of multiple IP subnets.
- `sameip`: Match traffic where source and destination IPs are identical.

D. Source & Destination Ports

- `any`: Matches any port.
- `80`: Specific port.
- `!21`: Any port except port 21.
- `1:1024`: Port range (ports 1 through 1024).
- `:1024`: Ports less than or equal to 1024.
- `1025:`: Ports greater than or equal to 1025.

E. Direction Operator

- `->`: One-way flow (Source $₩rightarrow$ Destination).
- `<>`: Bidirectional flow (Source $₩leftrightarrow$ Destination).

Rule Options (Enclosed in Parentheses)

General Options

- `msg:"..."`: Alert description string printed in logs.
- `sid:...`: Unique Snort Rule ID:
  - < 100: Reserved system rules.
  - 100 – 999,999: Pre-built official rules.
  - >= 1,000,000: Custom user-created rules.
- `rev:...`: Revision tracking integer for rule updates.
- `reference:...`: External documentation links (e.g., cve,CVE-XXXX).

Non-Payload Options (Header Checks)

- `id:...`: Match specific IP Identification header value.
- `flags:...`: Match TCP flag combinations (S, A, P, F, R, U).
- `dsize:...`: Match payload byte size (dsize:>500 or dsize:100<>200).
- `sameip`: Trigger if source IP equals destination IP.

Payload Options (Content Matching)

- `content:"..."`: Search payload for ASCII or HEX strings (content:"GET" or content:"|47 45 54|").
- `nocase`: Disable case sensitivity for preceding content.
- `fast_pattern`: Force Snort to use specified content rule for initial fast-filtering match when multiple content options are present.

Network Variables (snort.conf)

- `HOME_NET`: Protected internal network (e.g., 192.168.1.0/24).
- `EXTERNAL_NET`: Untrusted external network (usually !$HOME_NET or any).
- `RULE_PATH`: Directory path for rule files (/etc/snort/rules).

APPLICATION to cyber detective: comparing the three tools:

1. Wireshark: Packet Sniffing (Network Interface Level) -- COPIES

Wireshark uses an underlying packet capture library (libpcap on Linux/Mac or

Npcap on Windows) to tap directly into your Network Interface Card (NIC).

1. Promiscuous Mode: Normally, your NIC ignores network traffic that isn't addressed to your computer's MAC address. Wireshark puts the network card into *promiscuous mode*, instructing it to capture *every* packet passing through the physical wire or Wi-Fi channel, regardless of destination.
2. Kernel Tap: libpcap/Npcap injects a driver hook directly into the Operating System kernel. Before the network stack processes a packet, a raw copy is pulled directly out of kernel memory into Wireshark's buffer.
3. Passive Nature: Wireshark strictly reads copies of raw network frames. It does not stop, modify, or block traffic.

2. ZAP: Application Proxying (HTTP/Application Level) -PROXY/MODIFYING

Unlike Wireshark, ZAP is a True Proxy (a "Man-in-the-Middle" agent) sitting at the Application Layer (Layer 7). It does not tap raw network cards; instead, it pretends to be the server to your browser, and the browser to the real server.

1. Traffic Rerouting: You configure your web browser (or use ZAP's built-in browser) to route all HTTP/HTTPS traffic directly to ZAP's local port (e.g., 127.0.0.1:8080).
2. Explicit Handling: When you click a link, the browser opens a TCP socket to ZAP, sending the request directly to it. ZAP reads the request, logs/displays it, and then opens a *second*, separate connection to the actual target website on your behalf.
3. HTTPS Decryption (SSL Inspection): Because HTTPS encrypts data, ZAP generates its own local Root Certificate Authority (CA). When connecting to https://example.com, ZAP terminates the TLS connection, decrypts the payload in plain text for you to read, and creates a separate encrypted channel to the real server.

3. Snort: Inline Hooking & DAQ (Kernel/Network Stack Level)

Snort behaves as either a passive packet analyzer (like Wireshark) or an active inline blocker (IPS), depending on its Data Acquisition Module (DAQ) mode.

1. Passive Mode (pcap DAQ): Functions identically to Wireshark, using libpcap to read raw packet copies from the driver ring buffer without altering packet flow.
2. Inline / IPS Mode (afpacket or NFQ DAQ): * Snort uses Linux kernel modules like afpacket or Netfilter Queue (NFQ).

- Instead of just reading a copy, the Linux firewall (iptables or kernel network stack) literally passes active packets into Snort's queue buffer before delivering them to the destination or forwarding them across network interfaces.
- Snort evaluates the packet against its rules engine. If the packet passes, Snort releases it back to the kernel to continue its path. If a rule triggers a drop or reject, Snort instructs the kernel buffer to discard the memory block immediately, stopping the attack on the wire.

Summary of Primary Roles

Tool	Primary Purpose	Best Used For
Snort	Automated Guard & Monitor (IDS/IPS)	Running 24/7 on network gateways to automatically detect/block known attacks matching specific signatures.
Wireshark	Deep Packet Forensic Inspection	Opening captured raw traffic (.pcap) to inspect exact packet headers, reassemble TCP streams, and analyze non-web protocols (DNS, ARP, SMB).
OWASP ZAP	Active Web Application Testing	Penetration testing and security auditing of web apps, manipulating HTTP parameters, and inspecting decrypted HTTPS traffic in real time.

Snort's command line-based interface makes it hard to manually inspect interacts and packets

Wireshark is a visual forensics workbench, making it easy for humans to read interactions, packets, binary data, excels at displaying for forensic analysis

ZAP is different from the other two in that it is basically just a web proxy, just

http/https data not the full .pcap packets

A Real-World Workflow (How They Work Together)

Imagine a security incident on a company network:

1. Snort (The Detector): Triggers an automated alert: [1:1000001:1] Malicious SQL Injection Attempt Detected and saves the raw traffic log.
2. Wireshark (The Investigator): The incident responder opens Snort's .pcap log file inside Wireshark to reconstruct the entire TCP session, verify if the attack succeeded, and inspect what data was returned by the database.
3. OWASP ZAP (The Tester): The security team uses ZAP to reproduce the attacker's SQL injection attempt against a test instance of the web application to find and patch the underlying vulnerability.

Day 14: THM windows forensics pt1

Windows Registry: system's configuration data (Key(folder): value)

Root keys: all actively loaded user profiles

- HKEY_CURRENT_USER: specific settings to the current user
- HKEY_USERS: Profiles for all users on the machine
- HKEY_LOCAL_MACHINE: Settings for the entire computer system
- HKEY_CLASSES_ROOT: Which app opens which file extension

  Not a separate folder: merged view of:

  HKLM\Software\Classes(System-wide default file associations)

  HKLM\Software\Classes(Personal file association overrides)

- HKEY_CURRENT_CONFIG: info hardware profile used at startup

Majority of registry hives are located in C:\Windows\System32\Config:

- DEFAULT(HKEY_USERS)
- SAM(HKEY_LOCAL_MACHINE)
- SECURITY(HKEY_LOCAL_MACHINE)
- SOFTWARE(HKEY_LOCAL_MACHINE)
- SYSTEM(HKEY_LOCAL_MACHINE)

Hives containing user info: C:\Users\<username>/

- NTUSER.DAT(HKEY_CURRENT_USER when user logs in)
- USRCLASS.DAT(HKEY_CURRENT_USER\Software\CLASSES)

  Located in the directory(C:\Users\<username>\AppData\Local\Microsoft\Windows)

  Mounting: taking a dead,locked,binary file on the hard drive and plugging it into a specific socket in registry memory like HKLM(activates it to read/write)

  Path != where it mounts to

AmCache Hive: located in C:\Windows\AppCompat\Programs\Amcache.hve

➔ Save info on programs that were recently run on the system

Transaction log for each hive: a .LOG file in the same directory as hives(same name) – journal of changelog of the registry hive. Windows often uses these when writing data to registry hives. This means transaction logs have the most recent info that the hives might not have updated.

Registry backups: backups of hives located in C:\Windows\Systtem32\Config

- **Copied to the C:\Windows\System32\Config\RegBack every ten days**

Data Acquisition: Getting a copy of the registry hives in C:\Windows\System32\Config

Tools:

- **KAPE: Used to acquire registry data – lightweight, fast**
- **Autopsy: option to acquire data from both live systems or a disk image – full-featured digital forensics platform**
- **FTK Imager: extract files from a disk image or a live system**

  Obtain Protected Files option: only available for live streams, allows you to extrack all the registry hives to a location of your choosing(will not copy the Amcache.hve file, which is necessary)

Registry editor only works with live streams and can't load exported hives

Tools for viewing extracted files:

- **Registry Viewer: only loads one hive at a time, and can't take transaction logs into account**
- **Zimmerman's Registry Explorer: multiple hives simultaneously and add data from transaction logs into the hive, Bookmarks option(useful for forensics investigators)**
- **RegRipper: Takes hive as input and outputs a report that extracts data from some of the forensically important keys and values(text file)**

  Does not take transaction logs into account(not updated)

Registry keys for info:

Tools like Registry Explorer creates a virtual folder to help created already-mounted situations → allows using mounted paths like in live systems

- **OS Version:** SOFTWARE\Microsoft\Windows NT\CurrentVersion
- **Current control sets(two hives containing the machine's config data used for controlling system startup:**

  SYSTEM\Select\Current: points to the current control set

SYSTEM\Select\LastKnownGood: points to the last known good(backup) cinfig

SYSTEM\ControlSet001 or SYSTEM\ControlSet002 depending on what the above two keys say: gives the actual configs

- **Computer name:**
  SYSTEM\CurrentControlSet\Control\ComputerName\ComputerName
- **Time Zone Info:**

  SYSTEM\CurrentControlSet\Control\TimeZoneInformation

- **Network interfaces and Past Networks:**

  SYSTEM\CurrentControlSet\Services\Tcpip\Parameters\Interfaces

- **Autostart Programs:**

  NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Run

  NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\RunOnce

  SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce

  SOFTWARE\Microsoft\Windows\CurrentVersion\policies\Explorer\Run

  SOFTWARE\Microsoft\Windows\CurrentVersion\Run

- **Info about Services**

  `SYSTEM\CurrentControlSet\Services`

SAM Hive: Contains user account info, login info, group info

Mainly located in: `SAM\Domains\Account\Users`

🕵️‍♂️ FBI Cyber Detective Field Manual: Windows Registry

1. The Core Mindset

- **Live System = Mounted View:** The OS dynamically maps memory shortcuts (HKLM\SYSTEM\CurrentControlSet, HKCU).
- **Offline Analysis = Raw Hives:** On disk, you work directly with static hive files (SYSTEM, SOFTWARE, NTUSER.DAT) found in C:\Windows\System32\Config and C:\Users\<username>.

2. Quick Target Reference

Evidence Needed	Registry Path / Hive	Forensic Insight
Active Control Set	SYSTEM\Select\Current	Identifies whether ControlSet001 or ControlSet002 was actively running.
System Identity	SYSTEM\CurrentControlSet\Control\ComputerName	Establishes hostname for network tracking.
Time Alignment	SYSTEM\CurrentControlSet\Control\TimeZoneInformation	Critical for converting local evidence timestamps to UTC.
Network Footprint	SYSTEM\CurrentControlSet\Services\Tcpip\Parameters\Interfaces	Maps NICs, IP addresses, and past network adapters.
Malware Persistence	SOFTWARE & NTUSER.DAT ...\CurrentVersion\Run / RunOnce	Pinpoints programs configured to launch automatically on boot or user login.
User Evidence	SAM\Domains\Account\Users & NTUSER.DAT	Uncovers account SIDs, user login history, and user-specific activity.
Execution History	C:\Windows\AppCompat\Programs\Amcache.hve	Tracks recently executed binaries, hashes, file paths, and installation dates.

3. Tactical Toolkit

- **Acquisition (KAPE / FTK Imager):** Pulls raw, locked registry files directly from disks or live streams.
- **Triage (RegRipper / Autopsy):** Fast, automated scripts and big-picture dashboards to quickly spot obvious red flags.
- **Deep Surgical Analysis (Registry Explorer):** The gold standard for manual analysis. Automatically handles virtual paths (CurrentControlSet) and processes transaction logs.

4. Golden Forensic Rule

Raw Hive File (.DAT) + Transaction Logs (.LOG1 / .LOG2) = True System State

Never analyze a .DAT file in isolation. Uncommitted registry changes often live in transaction logs—failing to replay them means missing critical, time-sensitive evidence.

Day 15: Windows Forensics pt 2

Windows Recent files:

```
NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs
```

Different keys with file extensions: like .pdg, .jpg, .docx

Microsoft Office Recent Files:

```
NTUSER.DAT\Software\Microsoft\Office\VERSION(Version number)
```

Windows 'shell': folder display layout

```
USRCLASS.DAT\Local Settings\Software\Microsoft\Windows\Shell\Bags

USRCLASS.DAT\Local Settings\Software\Microsoft\Windows\Shell\BagMRU

NTUSER.DAT\Software\Microsoft\Windows\Shell\BagMRU

NTUSER.DAT\Software\Microsoft\Windows\Shell\Bags
```

Recently used files thru dialog MRUs(Most Recently Used): like open/save

```
NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\ComDlg32\OpenSavePIDlMRU
```

```
NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\ComDlg32\LastVisitedPidl
```

Paths typed in the windows explorer address bar or searches

```
NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\TypedPaths
```

```
NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\WordWheelQuery
```

UserAssist: recently launched apps(programs): excluding programs ran thru cmd

```
NTUSER.DAT\Software\Microsoft\Windows\Currentversion\Explorer\UserAssist\{GUID}\Count
```

ShimCache: mechanism used to keep track of app compatibility with the os – thus tracks all apps launched on the machine

```
SYSTEM\CurrentControlSet\Control\Session Manager\AppCompatCache
```

AmCache: stores data related to program executions

```
C:\Windows\appcompat\Programs\Amcache.hve
```

BAM(Background Activity monitor)/DAM(Desktop Activity Monitor)

```
SYSTEM\CurrentControlSet\Services\bam\UserSettings\{SID}
```

```
SYSTEM\CurrentControlSet\Services\dam\UserSettings\{SID}
```

Keep track of USB Keys plugged into a system

```
SYSTEM\CurrentControlSet\Enum\USBSTOR
```

```
SYSTEM\CurrentControlSet\Enum\USB
```

➔ **First/last times the device was connected:**

```
SYSTEM\CurrentControlSet\Enum\USBSTOR\Ven_Prod_Version\USBSerial#\Properties\{
83da6326-97a6-4088-9453-a19231573b29}\####
```

```
####:
```

Value	Information
0064	First Connection time
0066	Last Connection time
0067	Last removal time

Device name of the connected drive(also friendly name like USB)

```
SOFTWARE\Microsoft\Windows Portable Devices\Devices
```

Number of created users: Counting the user accounts with an RID greater than 1000 (or viewing the accounts listed under SAM\Domains\Account\Users\Names minus default built-in accounts):

1. NTUSER.DAT (User Profile Hive)

- **Location:** C:\Users\<Username>\NTUSER.DAT
- **Forensic Power:** Tracks individual, user-specific GUI actions and intent.
- **Abilities & Key Keys:**

    - **UserAssist**: Tracks GUI execution history, launch counts, and execution timestamps.
    - **RecentDocs**: Tracks recently opened or saved files by file extension.
    - **ComDlg32 (OpenSavePidlMRU / LastVisitedPidlMRU)**: Tracks files opened/saved in standard Windows dialog boxes and the directories accessed.
    - **TypedPaths & WordWheelQuery**: Logs exact folder paths typed into the File Explorer address bar and Windows search queries.
    - **RunMRU**: Logs commands typed into the Windows "Run" box (Win + R).

2. SYSTEM (Hardware, Drivers & OS Hive)

- **Location:** C:\Windows\System32\config\SYSTEM
- **Forensic Power:** Tracks system-level configurations, network interfaces, execution caches, and attached hardware.
- **Abilities & Key Keys:**
    - **USBSTOR & USB**: Stores vendor, product, and unique serial numbers of every USB mass storage device plugged into the machine.
    - **Properties\{83da6326...}**: Logs exact **First Connection**, **Last Connection**, and **Last Removal** timestamps for USB devices.
    - **MountedDevices**: Maps drive letters (e.g., E:) to volume GUIDs and hardware serial numbers.
    - **ShimCache (AppCompatCache)**: OS compatibility mechanism that tracks application metadata and execution history across the entire system.
    - **BAM / DAM (Background / Desktop Activity Monitor)**: Tracks active executable pathways and precise last-execution timestamps per user SID.

3. SOFTWARE (Installed Applications & OS Configuration Hive)

- **Location:** C:\Windows\System32\config\SOFTWARE
- **Forensic Power:** Maps system-wide software installations, auto-start persistence, and friendly device names.
- **Abilities & Key Keys:**
    - **Windows Portable Devices\Devices**: Links device serial numbers (from SYSTEM) to natural human-readable "Friendly Names" (e.g., "USB" or "SanDisk 64GB").
    - **Run / RunOnce**: Common malware persistence keys that force programs to launch automatically when Windows boots.
    - **CurrentVersion**: Holds OS build information, install dates, and registered owner details.

4. SAM (Security Account Manager Hive)

- **Location:** C:\Windows\System32\config\SAM
- **Forensic Power:** Stores local security accounts, password hashes (encrypted), and logon statistics.
- **Abilities & Key Keys:**
  - **Domains\Account\Users**: Lists all local user accounts.
  - **RID Analysis**: Distinguishes default/built-in accounts (RID < 1000) from user-created accounts (RID ≥ 1000).
  - **Account Metadata**: Tracks last logon time, password last set date, invalid login counts, and disabled account flags.

5. USRCLASS.DAT (User Shell Bag Hive)

- **Location:** C:\Users\<Username>\AppData\Local\Microsoft\Windows\USRCLASS.DAT
- **Forensic Power:** Tracks File Explorer window configurations and folder view settings.
- **Abilities & Key Keys:**
  - **Shell\Bags & BagMRU**: Records every folder directory a user opened and browsed in File Explorer—proving access to specific folders even if those folders or files were later deleted from the drive.

6. Amcache.hve (Application Compatibility & Malware Hunting Hive)

- **Location:** C:\Windows\appcompat\Programs\Amcache.hve
- **Forensic Power:** Dedicated OS execution logging database designed specifically for application compatibility tracking.
- **Abilities & Key Keys:**
  - **File Hashes ($SHA-1$)**: Stores cryptographic hashes of executed binaries, allowing investigators to instantly cross-reference executed files against threat intelligence databases (like VirusTotal).
  - **Full Execution Path & Timestamps**: Logs exact file paths, creation times, and first-execution times for applications, installers, and drivers.

7. SECURITY (Local Security Policy Hive)

- **Location:** C:\Windows\System32\config\SECURITY
- **Forensic Power:** Manages local security policies, user rights assignments, and domain cached credentials.
- **Abilities & Key Keys:**
  - **LSA Secrets**: Stores sensitive system-level credentials, service account passwords, and cached domain logon details.

The Ultimate Forensic Strategy:

1. Use **SAM** to prove *who* exists on the system.
2. Use **NTUSER.DAT** and **USRCLASS.DAT** to prove *what* that specific user targeted, opened, and browsed.
3. Use **SYSTEM**, **SOFTWARE**, and **Amcache** to prove *when* external hardware was attached and *what* executables were launched across the entire machine.